Universidade Estadual de Maringá
Centro de Tecnologia
Departamento de Informática

Professor: Nilton Luiz Queiroz Junior

Instruções:

- A aplicação deverá ser implementada na linguagem Racket;
- O trabalho vale de 0,0 a 10,0 e corresponde a 100% da nota da terceira avaliação periódica;
- Serão usados como critérios de avaliação:
  - A corretude do programa com relação ao que foi pedido;
  - A indentação do código;
  - A nomeação das variáveis e funções;
- O trabalho deverá ser entregue via classroom;
  - A data de entrega será definida em sala;
- O nome do arquivo enviado deve seguir o padrão: Nome_RAXXXXXX.zip;
  - O formato para submissão deve ser .zip, .rar, ou .tar;
- Não é necessário (mas recomenda-se) que sejam feitos casos de teste.
- O trabalho poderá ser feito em duplas;

Descrição: Deverá ser desenvolvida uma aplicação que seja capaz de contar a quantidade de cada uma das palavras dentro de um arquivo de texto, listar as N palavras que mais aparecem e suas respectivas contagens.

- As informações deverão ser armazenadas numa estrutura de memória principal;
- Serão passados três argumentos para o programa:
  - O arquivo de entrada:
    - Esse arquivo deverá ser lido, e todas as palavras existentes no arquivo deverão ser armazenadas com suas respectivas contagens.
  - Um valor inteiro N, sendo N maior que 0.
  - O arquivo de saída:
    - O programa deverá salvar um arquivo de texto, que irá exibir quais as N palavras que mais apareceram no arquivo, ordenadas de maneira decrescente (ou seja, a palavra que mais apareceu será a primeira a ser exibida, a segunda palavra que mais apareceu será a segunda a ser exibida, e assim por diante). Além disso, também deverá ser informado quantas vezes cada uma das N palavras apareceu, separando a palavra da contagem por um espaço em branco. Por fim, cada palavra deverá ser escrita em uma linha do arquivo.

      Obs 1: Caso exista empate na contagem, o desempate deverá ser feito pela ordem alfabética, fazendo com que as palavras com ordem léxica menor tenham preferencia na posição. **Deve ser levado em consideração a diferença entre letras maiúsculas e minúsculas, sendo:**

      A < B < … Z < … < a < b < …

      Obs 2: Caso o valor de N seja maior que a quantidade de palavras existentes no arquivo, todas as palavras devem ser escritas no arquivo de saída.

- Um arquivo contendo código para realizar o "parsing" dos argumentos passados para o programa, a leitura do arquivo de entrada (que gera uma única string) e a escrita do arquivo de saída (que recebe uma string para escrever no arquivo) está disponível no classroom da disciplina. Além disso, também estão disponíveis alguns arquivos de entrada e as contagens de cada palavra para aqueles arquivos.
    - Caso o grupo queira utilizar esse arquivo como base, será necessário implementar apenas a função `conta-palavras` que recebe o texto de entrada e o valor de N, e retorna a string contendo as N palavras que mais aparecem (no formato que será escrita no arquivo).
    - Caso o grupo queira realizar testes com outros arquivos, basta utilizar o seguinte comando no terminal:

```
racket base.rkt -i arquivo_de_entrada.txt -o arquivo_de_saida.txt -n N
```

    Onde:
        - arquivo_de_entrada.txt é o nome do arquivo de entrada com o texto.
        - arquivo_de_saída.txt é o nome do arquivo que será gerado com as n palavras q mais aparecem e suas contagens;
        - N é quantidade de palavras a serem escritas no arquivo de saída;
- Para a ordenação **não é permitido que a dupla utilize funções prontas em racket, ou seja, deverá ser implementado algum algoritmo de ordenação, ou alguma outra forma de selecionar os n primeiros elementos.**
- As demais funções para manipulação de listas, tais como concatenação, remoção, entre outras, podem ser usadas
- Dicas:

    Para facilitar o desenvolvimento, é sugerido que a dupla consulte as seguintes funções da linguagem racket (além, é claro, daquelas vistas em sala):
    - (string-split string): Quebra a string em uma lista de acordo com um separador (por padrão, o separador é dado pelos caracteres de espaço em branco). Link para a documentação:
      https://docs.racket-lang.org/reference/strings.html#%28def._%28%28lib._racket%2Fstring..rkt%29._string-split%29%29
    - (string-append str1 str2 …): Retorna a concatenação de todas as strings passadas como argumento. Link para a documentação:
      https://docs.racket-lang.org/reference/strings.html#%28def._%28%28quote._~23~25kernel%29._string-append%29%29
    - (string<? str1 str2 ...): Retorna verdadeiro se os argumentos estão em ordem crescente. Link para a documentação:
      https://docs.racket-lang.org/reference/strings.html#%28def._%28%28quote._~23~25kernel%29._string~3c~3f%29%29
    - E outras funções de comparação de string;

- Para que sejam escritas novas linhas no arquivo de texto, utilize o caractere '\n' na string de saída.
- Para facilitar a criação da string utilize a função format;
  - (format str v_1 v_2 ...): formata a string str colocando os valores v_i nos locais desejados para a string. Link para a documentação: https://docs.racket-lang.org/reference/Writing.html#%28def._%28%28quote._~23~25kernel%29._format%29%29

PROBLEMAS COM TRABALHOS COPIADOS E FEITOS COM IA:

Quem copiar o trabalho, da internet ou de outra dupla, terá o trabalho anulado (zerado), assim como o grupo que concedeu a cópia também.
Grupos que usarem IA para a implementação do código do trabalho também terão o trabalho anulado.